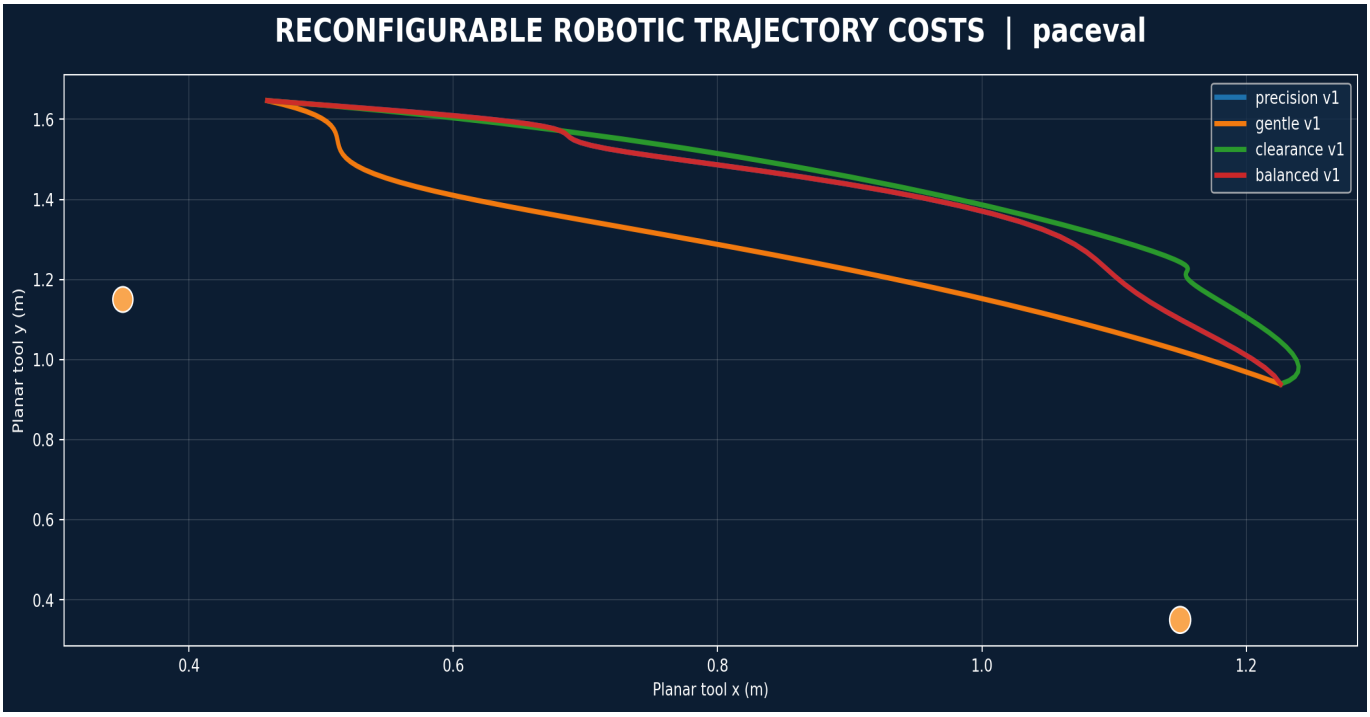
PACEVAL | SCIENTIFIC RESEARCH DEMONSTRATOR

Reconfigurable Robotic Trajectory Cost Functions

Complete-path scoring under alternative industrial priorities



Executed research result. Eight persistent paceval mathematical graphs, 16 deterministic complete-trajectory candidates of 51 samples each, four operating-priority models and one immutable formula revision. The original Linux SDK integration passed 6,783 scalar independent comparisons and 320 batch checks.

Research scope. Two-joint planar robot, analytical kinematics, finite-difference velocity, acceleration and jerk, two padded disk obstacles, and a host-managed sampled feasibility filter. Neither physical robot hardware nor MoveIt 2 was connected.

Cover image is a visualization of the actually selected output trajectories, not a photograph or a claim of full industrial robot certification.

1. Industrial use case and state of the art

Industrial robot applications routinely require trade-offs among accurate positioning, controlled motion, obstacle clearance and proximity to kinematic singularities. The preferred trade-off can change between precision assembly, delicate handling and movement through cluttered workspaces.

Current engineering practice

Modern motion planning systems compute kinematic states, check scene constraints, and rank or optimize candidate trajectories using chosen cost terms. MoveIt 2 exposes STOMP cost/validity callbacks for sampled full paths; CHOMP is an example of gradient-based motion optimization. Published work also studies jerk-oriented trajectories in industrial robotics. This feasibility study is a standalone numerical kernel, not a replacement for these planners.

The concrete demonstrated workflow

A deterministic C++ host generates 16 full joint-space candidates sharing identical start and end states. For each candidate it constructs 51 synchronized samples, computes finite-difference motion features and checks discrete joint and link-obstacle constraints. Persistent paceval execution graphs then evaluate the analytical joint kinematics and alternative cost functions; the host sums per-sample scores and chooses the lowest-cost feasible complete path.

Industrial question addressed

Can the same fixed candidate library be rescored under different, versioned mathematical preferences without rebuilding those formulas for each evaluation, and can its outputs be audited independently? The answer is positive for the defined numerical test, while actual industrial performance remains untested.

Technical sources: MoveIt STOMP task API, CHOMP tutorials and relevant modern jerk-oriented trajectory research are linked in the README and in this report references.

2. Mathematical model and assumptions

Planar two-link geometry: lengths l1 = 1.00 m and l2 = 0.78 m. Each candidate has N = 51 joint states, common endpoints, and fixed sample interval dt = 0.08 s (total sampled duration 4.0 s). Joint angles q1 and q2 are measured in radians.

Forward kinematics and singularity indicator

x(q) = l1 cos(q1) + l2 cos(q1 + q2)
y(q) = l1 sin(q1) + l2 sin(q1 + q2)
det(J) = l1 l2 sin(q2).

Versioned mathematical objective evaluated by paceval

For each sampled state the registered analytical graph computes a weighted sum of six terms: squared Cartesian tracking error, squared joint velocity, squared joint acceleration, squared joint jerk, a regularized clearance proximity term and a regularized singularity proximity term.

J(q) = wp ||p(q)-target||² + wv ||v||² + wa ||a||² + wj ||j||² + wo / (0.05 + c²) + ws / (0.1 + det(J)²).

The scalar full-trajectory objective is the finite sum dt times these sampled costs. The unnormalized weights and regularizers are explicitly synthetic engineering preferences, chosen for a controllable numerical demonstration. The combined score has illustrative units rather than a calibrated physical interpretation.

Discrete feasibility checks

The C++ host checks padded segment-to-disk squared-clearance c >= 0, finite-difference joint velocity magnitude <= 2.7 rad/s, acceleration magnitude <= 40 rad/s², and absolute Jacobian determinant >= 0.28 m². These are checks at sampled instants only, not continuous-time certificates. Two fixed padded disks and zero-thickness link centerlines define the idealized scene.

3. Persistent mathematical execution architecture

The original paceval_main.h and paceval_main.cpp are compiled from src/ and linked against a separately supplied, compatible paceval Linux shared library. The engine initializes once, creates persistent expression handles, reuses these for every candidate and releases them at shutdown.

Eight real reusable graph handles

Three handles compute x(q), y(q) and det(J). Five further handles compute the complete per-sample cost for: precision v1, gentle v1, clearance v1, balanced v1 and gentle v2. Each cost graph contains the kinematics and all weighted mathematical penalties. An update to the gentle-policy jerk coefficient is registered as a new handle rather than overwriting the previous one.

Explicit C++ / paceval division

Inside paceval: expression parsing, persistent graph computation, scalar calculation, and packed multi-input batch evaluation. **Inside C++:** deterministic joint paths, finite-difference features, closest point on each link segment, discrete feasibility, path summation, selection, CSV export and an independently implemented arithmetic oracle.

Scalar and batch verification

The application calls paceval_dGetComputationResult for every score and kinematic value. It also invokes paceval_dGetComputationResultExt on 64 packed samples for each of five cost graphs. Each result is checked against the independent native C++ formula using fixed predeclared tolerances; detailed diagnostics are retained in the audit CSV, not printed as an accuracy competition in this Whitepaper.

A path may be chosen by a different policy while all candidate geometry and feasibility constraints remain unchanged. This is the exact reconfigurability property tested in the experiment.

4. Executed test and independently checked results

The original paceval runtime executed a complete deterministic trajectory experiment. Unlike the earlier local 20-waypoint greedy demonstration, each alternative here is a whole 51-sample trajectory with common endpoints, and each policy evaluates the same fixed candidate set.

Executed and audited properties

- 8 persistent mathematical expression graphs
- 16 complete trajectory candidates, each with 51 samples
- 5 cost-policy versions, four distinct original operating priorities
- 6,783 independent scalar-value comparisons
- 320 independently checked packed SDK batch results
- 510 independently checked 80-digit forward-kinematics values
- 1/1 CMake/CTest integration test passed

Observed impact of changing the mathematical objective

precision v1 selects #3, gentle v1 selects #12, clearance v1 selects #4, balanced v1 selects #3, gentle v2 selects #12.

The precision objective selected candidate 3, the gentle objective candidate 12, and the clearance-priority objective candidate 4. The balanced model selected candidate 3. Both gentle-policy versions remain independently accessible, even when they select the same candidate.

These are rankings within a fixed, deliberately small candidate library. They are not globally optimal trajectories or certified collision-free robot programs.

5. Industrial path selection

The actual paceval-scored paths differ depending on the preference selected for the same robot, same endpoints and same obstacles.

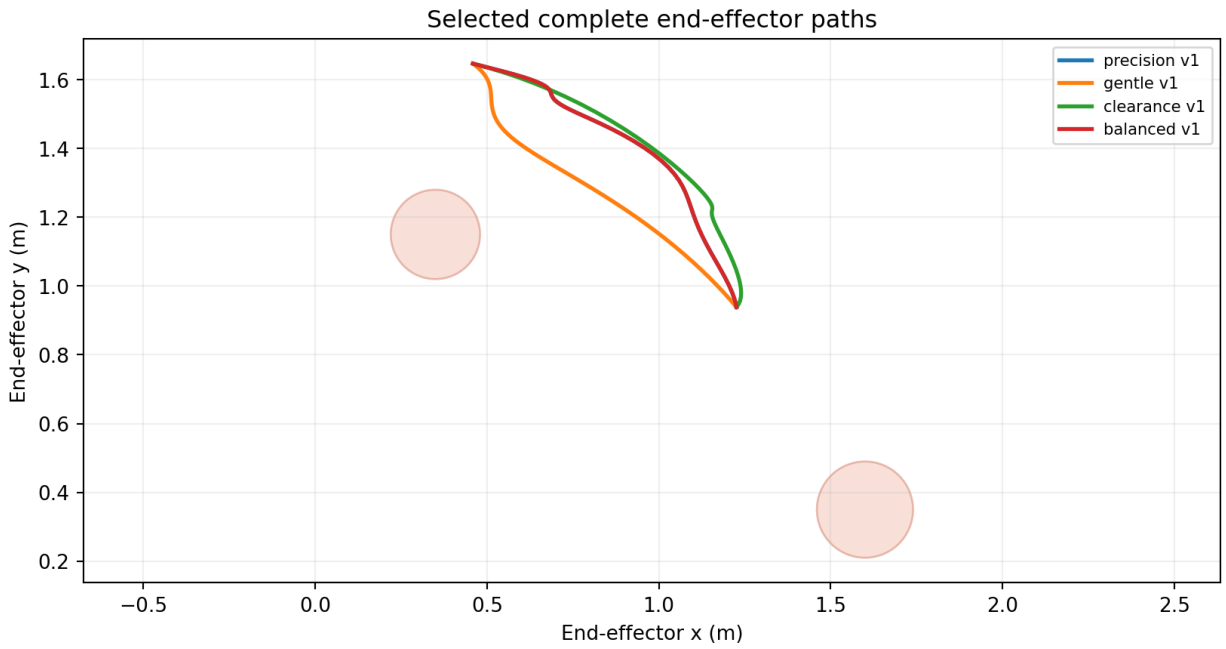


Figure 1. Selected complete Cartesian tool paths and two padded synthetic disks.

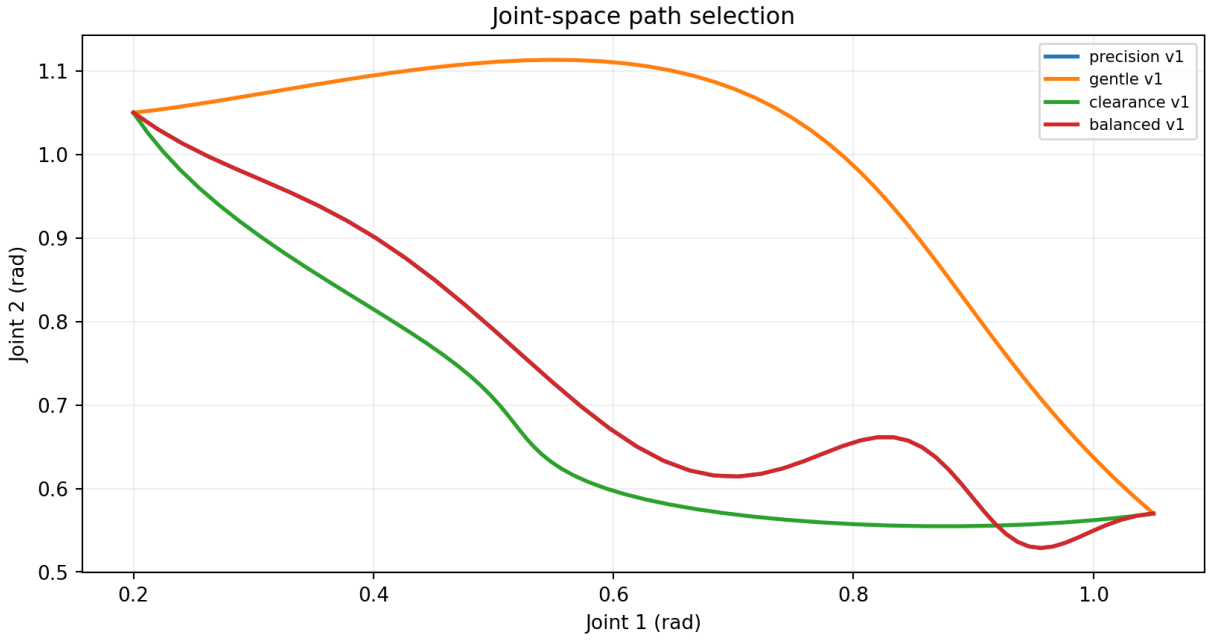


Figure 2. Joint-space curves for the corresponding selected trajectories.

6. Smoothness and sampled feasibility

All candidates are scored as complete sampled paths. Each accepted path satisfies the stated instantaneous finite-grid checks, while unsampled collisions and actuator dynamics remain out of scope.

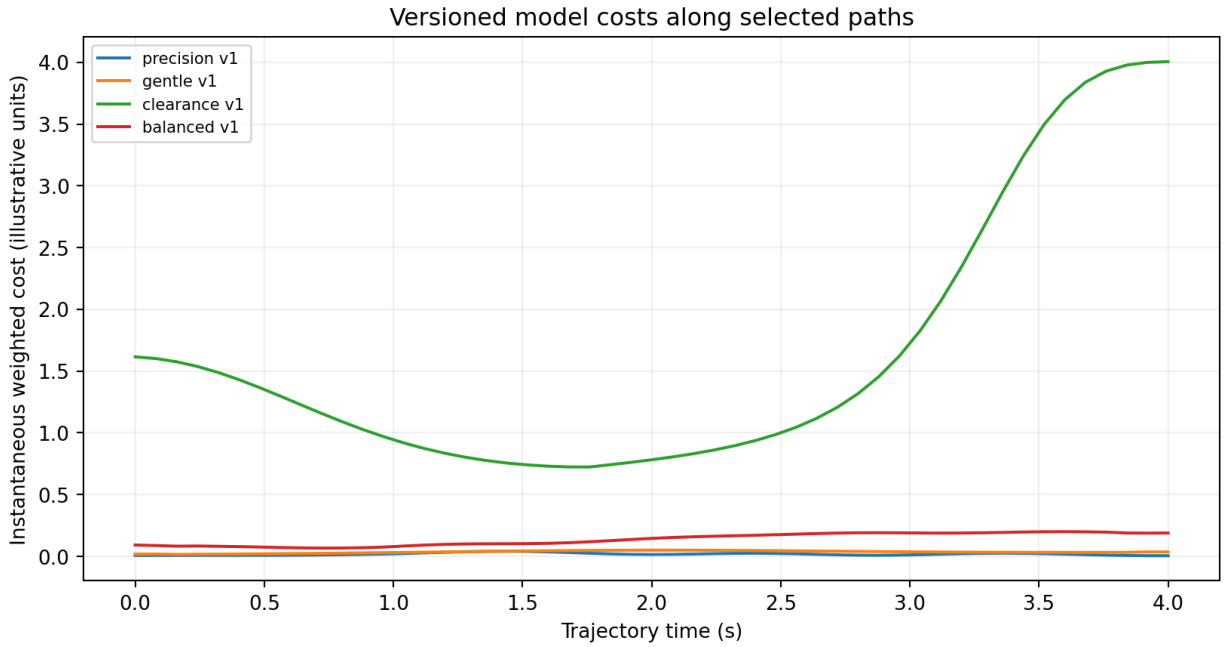


Figure 3. Weighted objective profiles along selected full candidate paths.

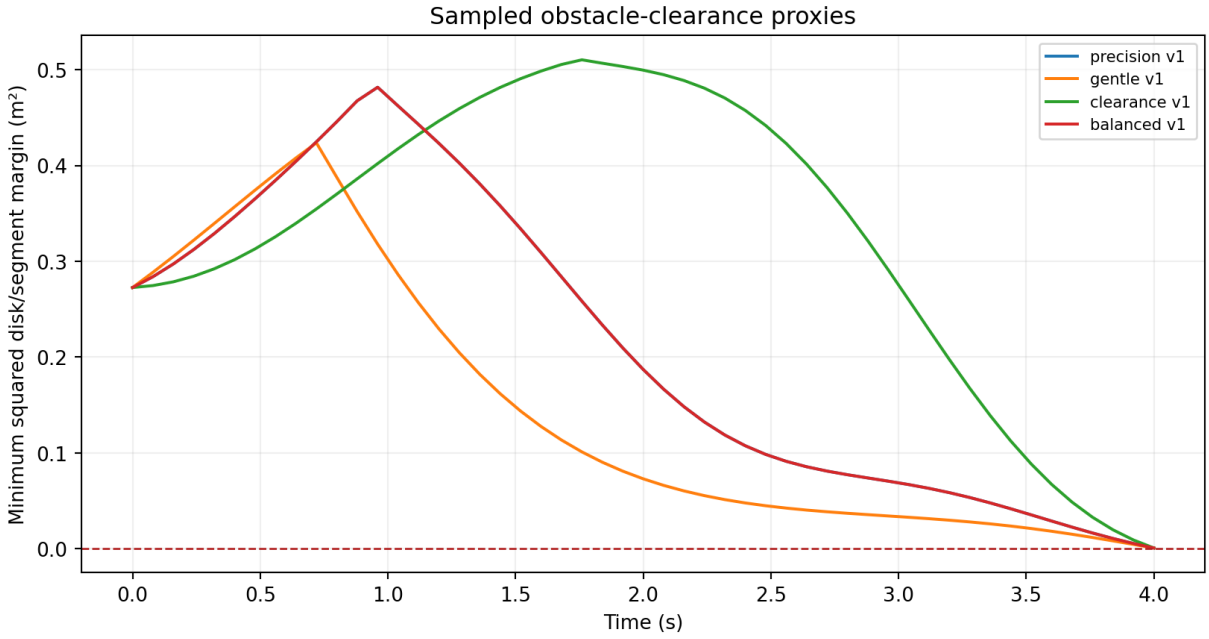


Figure 4. Minimum squared segment-to-disk margins at individual sampled instants.

7. Candidate ranking and motor positions

Reconfigurable preferences result in distinct score orderings. The model does not manipulate positions between samples to optimize the path; it selects from a frozen finite candidate library.

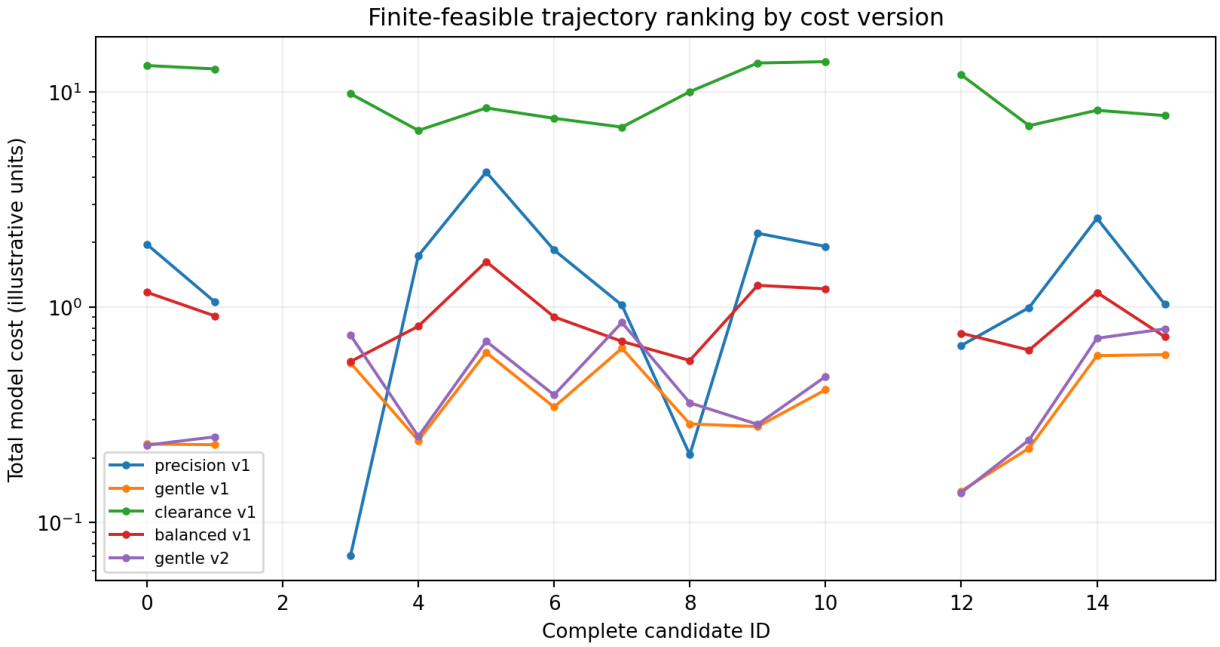


Figure 5. Feasible complete-candidate costs across the four priority models and revised version.

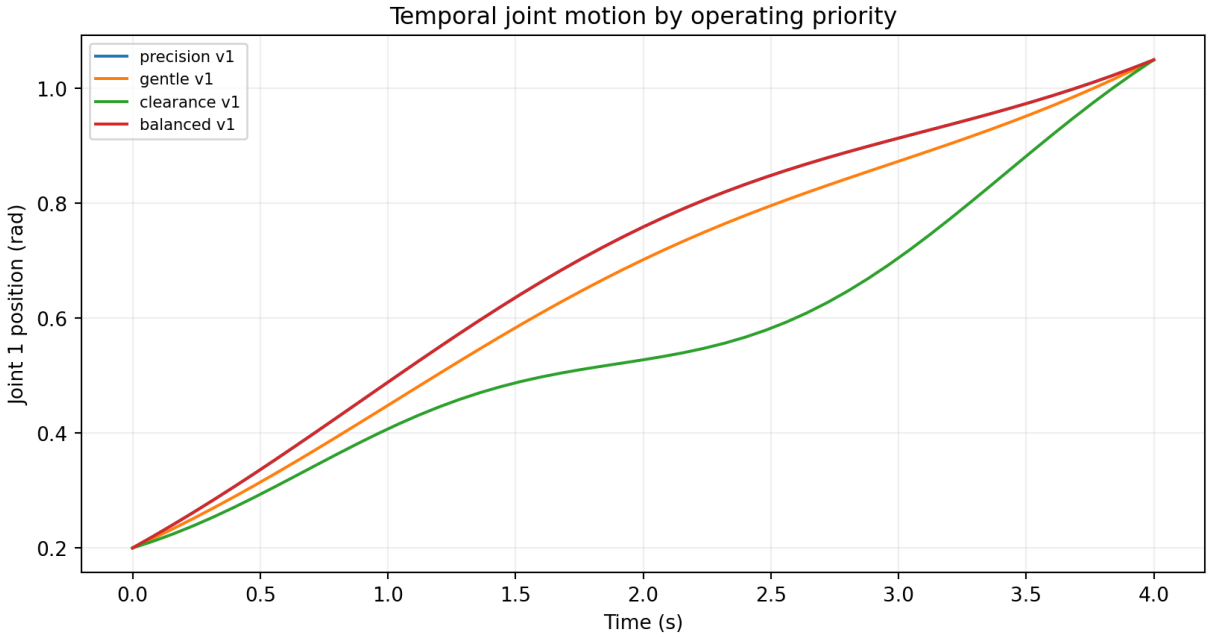


Figure 6. Temporal motion of joint 1 for the selected operating modes.

8. Immutable formula revision and cost map

Changing the jerk weighting creates a separate persistent paceval graph. This supports a traceable before-and-after comparison without invalidating the original expression.

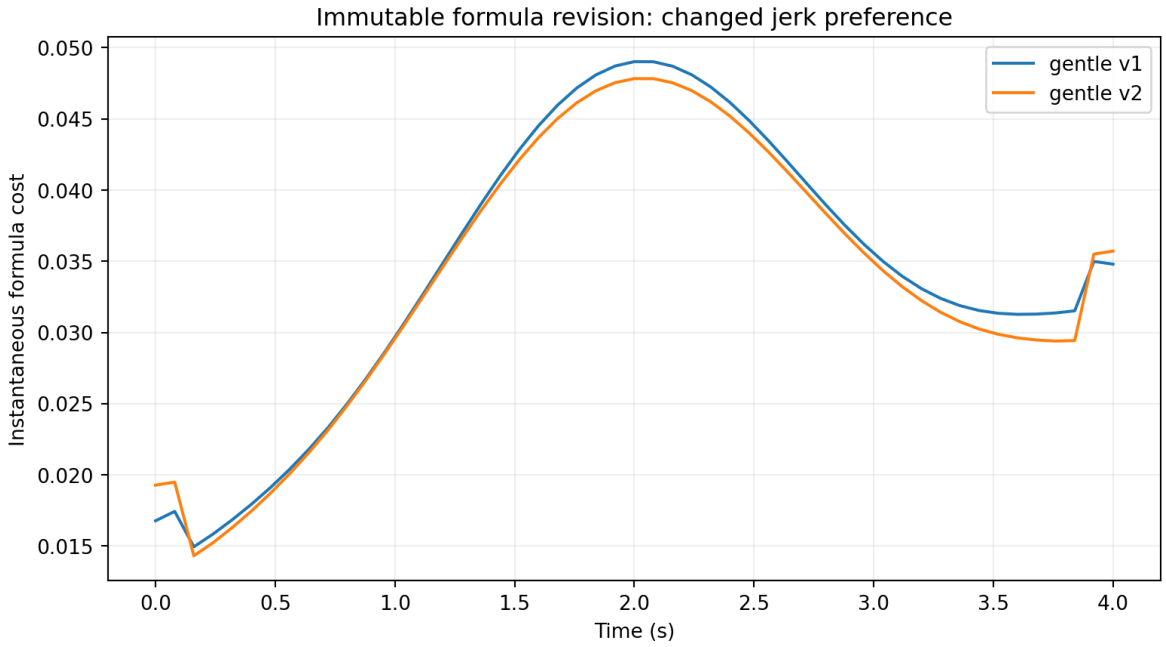


Figure 7. Gentle-mode v1 and v2 instantaneous cost evaluations.

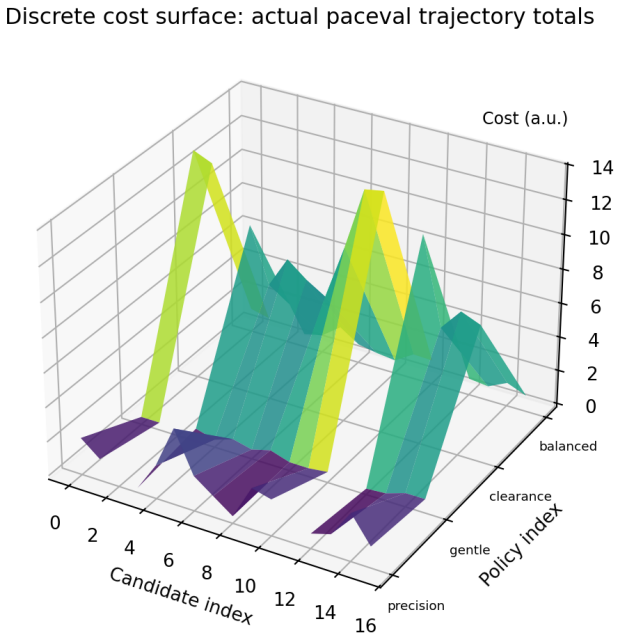


Figure 8. Discrete, masked 3D cost surface of the physically sampled finite candidate set.

9. Scientific limitations and industrial interpretation

What was successfully implemented

The program evaluates shared analytical robot kinematics and five genuinely versioned mathematical cost functions through the original paceval SDK. It selects among complete candidate paths under unchanged host-enforced sampled feasibility rules. Independent native and 80-decimal-digit calculations, CSV consistency checks and packed SDK evaluations passed on the published dataset.

What still requires independent engineering work

The model represents two planar zero-thickness links and exactly two padded circular obstacles. Sampled collision margins do not certify clearance between samples. Its simple joint velocity and acceleration limits do not model torque saturation, rigid-body dynamics, controller tracking errors or real sensor uncertainty. A finite candidate set cannot prove global optimality.

Relevant next engineering step

Integrate a documented, physically identified multi-joint robot with a real collision scene and the preferred MoveIt 2 trajectory-optimization workflow. Its STOMP task abstraction accepts a cost/validity callback, making it a natural integration target, but that step must include consistent collision checking, safety constraints, actual timing measurements and any required robot-specific verification.

Research conclusion

The additional WP031 contribution is controlled reconfiguration of full-path cost mathematics and demonstrably different complete-path choices. It extends the previous two-joint local waypoint study without claiming a new motion-planning algorithm or operational robot deployment.

10. Reproducibility and scientific references

Reproduction commands

Configure: cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so. Build: cmake --build build -j. Test: ctest --test-dir build --output-on-failure. Data: ./build/trajectory_costs --out examples. Independent audits: python3 src/verify_results.py and python3 src/high_precision_reference.py. Figures: python3 src/make_figures.py.

Audited files

README.md; src/trajectory_costs.cpp; src/paceval_main.h; src/paceval_main.cpp; src/CMakeLists.txt; examples/trajectory_candidates.csv; examples/selected_trajectories.csv; examples/selected_path_points.csv; examples/formula_registry.csv; examples/numeric_audit.csv; results/high_precision_audit.json; results/results_audit.json; figures/*.png and *.pdf.

Scientific and technical references

MoveIt 2 STOMP composable task API: https://github.com/moveit/moveit2/blob/main/moveit_planners/stomp/include/stomp_moveit/stomp_moveit_task.hpp

MoveIt 2 CHOMP planning documentation: https://github.com/moveit/moveit2_tutorials/blob/main/doc/how_to_guides/chomp_planner/chomp_planner_tutorial.rst

Time-jerk optimal trajectory planning (2024): https://doi.org/10.1002/rob.22298

paceval official project and API: https://github.com/paceval/paceval

The report presents synthetic engineering scores for mathematical reproducibility. No real robot, MoveIt 2 service, actual industrial cycle or performance benefit has been experimentally measured.